SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Artificial Intelligence and Data Science

ASSESSMENT TOOL 3 – CODING CHALLENGE

Course Code:	DSA03	**Course Title:**	Natural Language Processing
CO Assessed:	CO1 – Imitation (L1)	**Assessment:**	Assessment Tool 3 – Coding Challenge
Weightage:	30% of CIA		
CO5 Statement:	Analyse NLP models, regular expression patterns, finite state automata, morphological processing techniques and to interpret language processing. (BL-3)		
Student Name:	M. SAHASRA	**Reg. No.:**	192425317
Section / Batch:	2024-2028	**Date:**	15-07-2026

Code of Conduct

I M. SAHASRA 192425317 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: M. Sahasra

Instructions

- Develop a complete and modular Python program that meets all the functional requirements specified in the coding challenge using appropriate algorithms, data structures, and programming practices.
- Test the program using multiple valid and invalid test cases, and clearly display the input, processing, and output to verify the correctness of the implementation.
- Follow standard coding practices by using meaningful variable names, proper indentation, comments, exception handling (where applicable), and upload the source code, sample outputs, and README file to the designated GitHub repository.
- Duration: 40 minutes.

Section / Topic	Focus	Q Nos
Regular Expression Pattern Matching	Regular Expressions, Basic Regular Expression Patterns	1
Finite State Automata Implementation	Finite State Automata, Models and Algorithms	2
Advanced Regular Expression Search	Regular Expressions, Basic Regular Expression Patterns	3

Annexure A – Coding Challenge

Section 1: Intelligent Email and Password Validator using Regular Expressions

1. **Problem Statement:**

A software company is developing a user registration system. Before creating an account, the system must validate user credentials based on predefined security policies.

Develop a Python application that validates:
Email Address
Password
Mobile Number

using Regular Expressions.

Validation Rules
Email
Must start with an alphabet.
May contain letters, digits, '.', '_' before '@'.
Domain should contain only alphabets.
Valid extensions: .com, .org, .edu, .net, .in
Password
Minimum 8 characters
At least one uppercase letter
At least one lowercase letter
At least one digit
At least one special character (@,#,$,%,&,!)
Mobile Number
Must contain exactly 10 digits.
First digit should be between 6–9.
Expected Output

Display:

Valid Email / Invalid Email
Strong Password / Weak Password
Valid Mobile Number / Invalid Mobile Number

Section 2: Design a Finite State Automata (FSA) Simulator

2. **Problem Statement**

Develop a Python-based Deterministic Finite Automata (DFA) simulator.

The program should:

- Accept the DFA description
    - States
    - Input alphabet
    - Transition table
    - Initial state
    - Final state(s)
- Accept multiple input strings.
- Simulate state transitions.
- Display whether the string is Accepted or Rejected.

Example

Language:
Strings ending with "ab"

Input
abaab

Output
Transition Path:
q0 → q1 → q0 → q1 → q2
Accepted

Section 3: Smart Pattern Matching Engine using Regular Expressions

3. Problem Statement

Develop a Python-based text search engine that performs pattern matching using Regular Expressions.

The system should support:
Word Search
Prefix Search
Suffix Search
Date Search
Phone Number Search
Hashtag Search
Mention (@username) Search

Example Input
Meeting on 12/09/2026
Call 9876543210
#NLP
@OpenAI
natural language processing

User Menu
1.Search Date
2.Search Phone Number
3.Search Hashtag
4.Search Mention
5.Search Prefix
6.Search Suffix

Expected Output
Display all matching patterns based on user selection.

Rubrics for Calculation

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	20%	Clearly understands problem constraints, edge cases, and competitive requirements	Understands problem with minor gaps in constraints	Partial understanding; misses key constraints	Misinterprets problem or constraints
Algorithm	25%	Selects optimal algorithm with best time and space complexity for competitive constraints	Uses efficient algorithm with minor scope for improvement	Uses basic/inefficient approach	No clear algorithmic approach
Code Implementation	20%	Writes correct, efficient, and clean code that passes all constraints	Code works with minor errors or inefficiencies	Partially correct code; fails for some cases	Code does not execute or gives incorrect results
Competitive Performance	20%	Solves problem within time limits and handles large inputs effectively	Solves within acceptable time with minor delays	Struggles with time limits or larger inputs	Unable to solve within time constraints
Test Case Handling	15%	Handles all test cases including edge and hidden cases	Handles most test cases	Misses important edge cases	Fails on multiple test cases

Q. No. / Criteria	Problem Understanding	Algorithm	Code Implementation	Competitive Performance	Test Case Handling	Sub. Total	Total %
1							
2							
3							

Faculty In-charge	Course Coordinator	Program Director
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

1)

Aim:
To develop a Python program using Regular Expressions that validates an email address, password, and mobile number according to the given security rules.

Problem Understanding:
A software company requires an automated validation system during user registration. The system should check whether the entered email, password, and mobile number satisfy predefined security rules using Regular Expressions and display appropriate validation messages.

Algorithm:

1. Start the program.
2. Import the re module.
3. Read the email, password, and mobile number from the user.
4. Create regular expression patterns for email validation.
5. Create regular expression patterns for password validation.
6. Create regular expression patterns for mobile number validation.
7. Compare the user inputs with their respective patterns.
8. Display Valid Email or Invalid Email.
9. Display Strong Password or Weak Password.
10. Display Valid Mobile Number or Invalid Mobile Number.
11. Stop the program.

Python Code:

```python
import re

email = input("Enter Email: ")
password = input("Enter Password: ")
mobile = input("Enter Mobile Number: ")

email_pattern = r'^[A-Za-z][A-Za-z0-9._]*@[A-Za-z]+\.(com|org|edu|net|in)$'

password_pattern = r'^(?=.*[A-Z])(?=.*[a-z])(?=.*\d)(?=.*[@#$%&!]).{8,}$'

mobile_pattern = r'^[6-9]\d{9}$'

if re.match(email_pattern, email):
    print("Valid Email")
else:
    print("Invalid Email")

if re.match(password_pattern, password):
    print("Strong Password")
else:
    print("Weak Password")

if re.match(mobile_pattern, mobile):
    print("Valid Mobile Number")
else:
    print("Invalid Mobile Number")
```

Sample Input:

```
Enter Email: sahasra123@gmail.com
Enter Password: Sahasra@123
Enter Mobile Number: 9876543210
```

Sample Output:

```
Valid Email
Strong Password
Valid Mobile Number
```

Result

The program successfully validates the email address, password, and mobile number using Regular Expressions and displays the appropriate validation status.

---

2)

Aim:

To develop a Python program that simulates a Deterministic Finite Automata (DFA) and checks whether the given input string is accepted or rejected based on the defined state transitions.

Problem Understanding:

A Deterministic Finite Automata (DFA) is used to recognize patterns in strings. The program should define states, transitions, an initial state, and final state(s). It should process an input string symbol by symbol, display the transition path, and determine whether the string is accepted or rejected.

Algorithm:

1. Start the program.
2. Define the DFA states.
3. Define the input alphabet.
4. Create the transition table.
5. Specify the initial state.
6. Specify the final (accepting) state.
7. Read the input string from the user.
8. Initialize the current state with the initial state.
9. Read each character of the input string one by one.
10. Change the current state according to the transition table.
11. Store every visited state to display the transition path.
12. After processing the entire string, check whether the current state is a final state.

13. If it is a final state, display Accepted.
14. Otherwise, display Rejected.
15. Stop the program.

Python Code:

```python
# DFA to accept strings ending with "ab"

transitions = {
    ('q0', 'a'): 'q1',
    ('q0', 'b'): 'q0',

    ('q1', 'a'): 'q1',
    ('q1', 'b'): 'q2',

    ('q2', 'a'): 'q1',
    ('q2', 'b'): 'q0'
}

start_state = 'q0'
final_state = 'q2'

string = input("Enter Input String: ")

current = start_state
path = [current]

for ch in string:
    if (current, ch) in transitions:
        current = transitions[(current, ch)]
        path.append(current)
    else:
        print("Invalid Input")
        exit()

print("Transition Path:")
print(" -> ".join(path))

if current == final_state:
    print("Accepted")
else:
    print("Rejected")
```

Sample Input:

```
Enter Input String: abaab
```

Sample Output:

```
Transition Path:
q0 -> q1 -> q2 -> q1 -> q1 -> q2
Accepted
```

Result

The program successfully simulates a Deterministic Finite Automata (DFA), displays the transition path, and correctly determines whether the given input string is accepted or rejected.

---

3)

Aim:

To develop a Python program that performs various pattern searches such as date, phone number, hashtag, mention, prefix, and suffix using Regular Expressions.

Problem Understanding:

The program acts as a simple text search engine. It extracts different patterns from a given text using Regular Expressions and displays the matching results based on the user's menu selection.

Algorithm:

1. Start the program.
2. Import the re module.
3. Store the input text.
4. Display the search menu.
5. Read the user's choice.
6. According to the selected option, apply the corresponding Regular Expression.
7. Search for the required pattern in the text.
8. Display all matching results.
9. If no match is found, display an appropriate message.
10. Stop the program.

Python Code:

```
import re

text = """
Meeting on 12/09/2026
Call 9876543210
#NLP
@OpenAI
natural language processing
"""

print("1.Search Date")
print("2.Search Phone Number")
print("3.Search Hashtag")
print("4.Search Mention")
print("5.Search Prefix")
print("6.Search Suffix")

choice = int(input("Enter Choice: "))

if choice == 1:
    print(re.findall(r'\d{2}/\d{2}/\d{4}', text))

elif choice == 2:
    print(re.findall(r'[6-9]\d{9}', text))

elif choice == 3:
    print(re.findall(r'#\w+', text))

elif choice == 4:
    print(re.findall(r'@\w+', text))

elif choice == 5:
    prefix = input("Enter Prefix: ")
    print(re.findall(r'\b' + prefix + r'\w*', text, re.IGNORECASE))

elif choice == 6:
    suffix = input("Enter Suffix: ")
    print(re.findall(r'\w*' + suffix + r'\b', text, re.IGNORECASE))

else:
    print("Invalid Choice")
```

Sample Input:

```
1.Search Date
2.Search Phone Number
3.Search Hashtag
4.Search Mention
5.Search Prefix
6.Search Suffix
Enter Choice: 1
```

Sample Output:

```
['12/09/2026']
```

Sample Input:

```
1.Search Date
2.Search Phone Number
3.Search Hashtag
4.Search Mention
5.Search Prefix
6.Search Suffix
Enter Choice: 5
Enter Prefix: nat
```

Sample Output:

```
['natural']
```

Sample Input:

```
1.Search Date
2.Search Phone Number
3.Search Hashtag
4.Search Mention
5.Search Prefix
6.Search Suffix
Enter Choice: 3
```

Sample Output:

```
['#NLP']
```

Result:

The program successfully performs different types of pattern matching using Regular Expressions and displays the required search results based on the user's selection.